O Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de Semicondutores (PADIS)

GUSTAVO FELIPE
LAURO BRIZIDIO

UNIVERSALIZATION OF IC DESIGN FROM CASS
MULTIPLEXADOR 4:1

MANAUS - AM
2026

SUMÁRIO

1. Introdução..... 3
2. Objetivos.....4
2.1. Objetivos Específicos.....4
3. Materiais e Softwares Utilizados..... 4
4. Resultados Experimentais..... 4
4.1. Código Verilog.....4
4.2. Simulação - GTK Wave.....7
Conclusão.....9
6. Referências Bibliográficas.....10

1. Introdução

Os circuitos digitais combinacionais são fundamentais no projeto de sistemas eletrônicos modernos. Entre esses circuitos, o multiplexador (MUX) desempenha um papel essencial ao permitir a seleção de um entre diversos sinais de entrada para ser encaminhado à saída, de acordo com sinais de controle.

Um multiplexador 4-para-1 é um dispositivo digital que possui quatro entradas de dados, duas linhas de seleção e uma única saída. Dependendo do valor das linhas de seleção, uma das entradas é direcionada para a saída do circuito.

Neste trabalho foi desenvolvido e simulado um multiplexador 4:1 utilizando a linguagem de descrição de hardware SystemVerilog. A simulação foi realizada com a ferramenta Icarus Verilog e os sinais gerados foram analisados utilizando o visualizador de formas de onda GTKWave.

2. Objetivos

O objetivo desta atividade foi:

- Desenvolver e simular um multiplexador 4:1 utilizando a linguagem de descrição de hardware SystemVerilog, verificando seu funcionamento por meio de simulação digital e análise das formas de onda geradas.

2.1. Objetivos Específicos

- Implementar um multiplexador 4:1 utilizando estrutura modular em SystemVerilog.
- Utilizar um parâmetro para tornar o circuito genérico em relação à largura dos dados.
- Desenvolver um testbench para gerar estímulos de entrada e verificar o comportamento do circuito.
  Realizar a simulação funcional utilizando o simulador Icarus Verilog.
- Analisar as formas de onda obtidas na simulação utilizando a ferramenta GTKWave.
- Validar o funcionamento do multiplexador conforme a tabela de verdade do circuito.

3. Materiais e Softwares Utilizados

Para a realização desta atividade, foram utilizados os seguintes recursos:

- Icarus Verilog
- GTK Wave
- Computador Linux - Ubuntu 22.04.5 LTS

4. Resultados Experimentais

4.1. Código Verilog

```
     module mux_4_to_1
#(
    parameter DATA_WIDTH = 8
)
(
    input  logic [DATA_WIDTH-1:0] D0,
    input  logic [DATA_WIDTH-1:0] D1,
    input  logic [DATA_WIDTH-1:0] D2,
    input  logic [DATA_WIDTH-1:0] D3,
    input  logic [1:0] S,
    output logic [DATA_WIDTH-1:0] Y
);

always @(*) begin
    case(S)
        2'b00: Y = D0;
        2'b01: Y = D1;
        2'b10: Y = D2;
```

```
        2'b11: Y = D3;
        default: Y = '0;
    endcase
end

endmodule
~
```

Figura 1 - Codigo Verilog Mux 4_to_1
Fonte: Autores (2026)

```
`timescale 1ns/1ps

module mux_tb;

parameter DATA_WIDTH = 8;

// Entradas do DUT
logic [DATA_WIDTH-1:0] D0;
logic [DATA_WIDTH-1:0] D1;
logic [DATA_WIDTH-1:0] D2;
logic [DATA_WIDTH-1:0] D3;

logic [1:0] S;

// Saída
logic [DATA_WIDTH-1:0] Y;

// Instância do DUT (Device Under Test)
mux_4_to_1 #(
    .DATA_WIDTH(DATA_WIDTH)
) dut (
    .D0(D0),
    .D1(D1),
    .D2(D2),
    .D3(D3),
    .S(S),
    .Y(Y)
);

initial begin

    // arquivo de waveform
    $dumpfile("mux.vcd");
    $dumpvars(0, mux_tb);

    // valores iniciais
```

```
    D0 = 8'h11;
    D1 = 8'h22;
    D2 = 8'h33;
    D3 = 8'h44;

    // teste seleção
    for (int i = 0; i < 4; i++) begin

        S = i;
        #10;

        case (S)
            2'b00: if (Y !== D0) $error("Erro: Y != D0");
            2'b01: if (Y !== D1) $error("Erro: Y != D1");
            2'b10: if (Y !== D2) $error("Erro: Y != D2");
            2'b11: if (Y !== D3) $error("Erro: Y != D3");
        endcase

    end

    $display("Teste finalizado.");
    $finish;

end

endmodule
```

Figura 2 - Código Verilog Mux TestBench.
Fonte: Autores (2026)

Após a implementação do módulo do multiplexador em **SystemVerilog** e a criação do respectivo testbench para verificação funcional do circuito, foi realizada a compilação dos arquivos utilizando o simulador **Icarus Verilog**.

Para compilar os arquivos e gerar o executável da simulação, foi utilizado o seguinte comando no terminal:

```
iverilog -g2012 mux_4_to_1.sv mux_tb.sv -o mux.out
```

Após a compilação, o arquivo executável gerado foi utilizado para executar a simulação do circuito digital, produzindo os dados necessários para análise do comportamento do multiplexador.

Após a compilação do código, foi executado o arquivo gerado pelo simulador

utilizando o interpretador do **Icarus Verilog**. Para isso, foi utilizado o seguinte comando no terminal:

```
vvp mux.out
```

Esse comando executa o arquivo `mux.out`, gerado durante o processo de compilação, iniciando a simulação do circuito digital descrito em **SystemVerilog**. Durante a execução da simulação, o testbench aplica estímulos às entradas do multiplexador e registra a evolução dos sinais ao longo do tempo.

Como resultado da simulação, é gerado um arquivo de forma de onda no formato **Value Change Dump (VCD)**, denominado `mux.vcd`. Esse arquivo contém o histórico das transições dos sinais do circuito ao longo do tempo, permitindo a análise do comportamento funcional do sistema.

Para visualizar os sinais gerados durante a simulação, foi utilizado o visualizador de formas de onda **GTKWave**, executado com o seguinte comando:

```
gtkwave mux.vcd
```

4.2. Simulação - GTK Wave

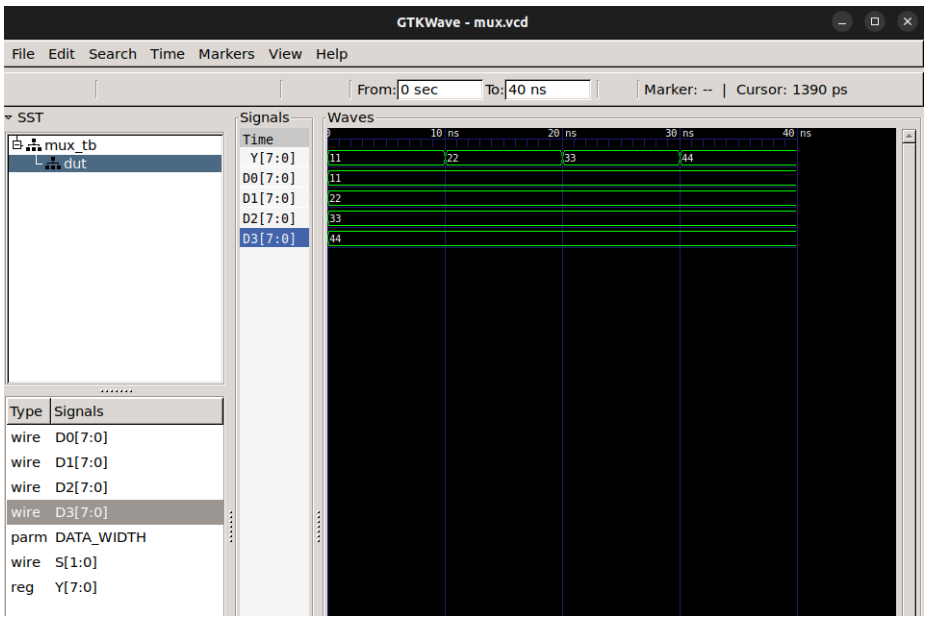

Figura 3 - GTKWave simulação Y[7:0] e as entradas D0:D3
Fonte: Autores (2026)

No ambiente do GTKWave foi possível observar os sinais correspondentes às entradas do multiplexador (D0, D1, D2, D3), às linhas de seleção (S0, S1) e à saída (Y). A análise das formas de onda permite verificar o comportamento funcional do multiplexador 4:1 descrito em SystemVerilog.

Durante a simulação, o testbench altera sequencialmente os valores das linhas de seleção (S1 e S0), o que provoca a mudança da entrada de dados conectada à saída.

Observa-se que:

- Quando S1S0 = 00, a saída Y assume o valor presente na entrada D0.
- Quando S1S0 = 01, a saída Y passa a refletir o valor da entrada D1.
- Quando S1S0 = 10, a saída Y corresponde ao valor da entrada D2.
- Quando S1S0 = 11, a saída Y assume o valor da entrada D3.

Conclusão

Neste trabalho foi desenvolvido um multiplexador 4:1 utilizando a linguagem SystemVerilog. O circuito foi descrito de forma parametrizável, permitindo alterar a largura dos dados de entrada conforme necessário.

A validação do circuito foi realizada por meio de simulação com o simulador Icarus Verilog, e as formas de onda foram analisadas utilizando a ferramenta GTKWave. Os resultados obtidos confirmaram que o circuito implementado atende ao comportamento esperado de um multiplexador 4:1.

A atividade permitiu consolidar conhecimentos sobre modelagem de circuitos digitais, uso de linguagens de descrição de hardware e verificação funcional através de simulação.

6. Referências Bibliográficas

UNIC-CASS. **uniccass-icdesign-tools**: ambiente com ferramentas open-source para projeto de circuitos integrados. GitHub. Disponível em: https://github.com/unic-cass/uniccass-icdesign-tools. Acesso em: 5 mar. 2026.

PALNITKAR, Samir. *Verilog HDL: A Guide to Digital Design and Synthesis*. 2. ed. Upper Saddle River: Prentice Hall, 2003.

ICARUS VERILOG. *Icarus Verilog Compiler*. Disponível em: http://iverilog.icarus.com. Acesso em: 5 mar. 2026.

GTKWAVE. *GTKWave Waveform Viewer*. Disponível em: http://gtkwave.sourceforge.net. Acesso em: 5 mar. 2026.